

# Probabilistic Machine Learning

---

## Lecture 01: Introduction

Prof. Dr. Michael Granitzer

Chair of Data Science · University of Passau

## Speaker notes

Reworked 2026-09-08 along docs/reviews/PML-L01-REWORK-2026-09.md: the lecture is a guided progression prediction → uncertainty → its uses → its sources → probability as the language → what the course enables → the road ahead. Formal vocabulary is deferred to the final section on purpose.

TODO (needs input): the title slide should carry the University of Passau logo and the intended affiliation line (ITU fellowship). Neither asset nor wording exists in this repository — add the logo to figures/ and the wording here.



# Why Probability?

---

**Guiding Question:** Two models both say "fraud". What would you need to know before acting on either — and why does a label not tell you?

# Machine Learning Gives Us Predictions

Three are flagged as fraud  
and the three labels are identical

|                                     |                                     |                                          |                                        |
|-------------------------------------|-------------------------------------|------------------------------------------|----------------------------------------|
| Café Central<br>€4.20<br>legitimate | Supermarket<br>€63.10<br>legitimate | Electronics Online<br>€1,299.00<br>fraud | Petrol Station<br>€78.40<br>legitimate |
| Gift Cards ×12<br>€600.00<br>fraud  | Bookshop<br>€24.90<br>legitimate    | Hotel Booking<br>€410.00<br>fraud        | Pharmacy<br>€12.75<br>legitimate       |

## Speaker notes

Stepped visual, then discussion stepper. Click 1: the model stamps every transaction — five legitimate, three fraud. Click 2: the legitimate ones recede and the three flagged cases stand alone. Pause here: a €1,299 electronics order, twelve gift cards, and a hotel booking are obviously *different situations*, yet the model's output for them is one identical word.

Then reveal the questions one at a time and let the room answer before the next appears. Students should feel that a label gives them nothing to separate a near-certain case from a coin flip. Do not name the missing element — the next slide lets them discover it.



# Machine Learning Gives Us Predictions

Three are flagged as fraud  
and the three labels are identical

|                                     |                                     |                                          |                                        |
|-------------------------------------|-------------------------------------|------------------------------------------|----------------------------------------|
| Café Central<br>€4.20<br>legitimate | Supermarket<br>€63.10<br>legitimate | Electronics Online<br>€1,299.00<br>fraud | Petrol Station<br>€78.40<br>legitimate |
| Gift Cards ×12<br>€600.00<br>fraud  | Bookshop<br>€24.90<br>legitimate    | Hotel Booking<br>€410.00<br>fraud        | Pharmacy<br>€12.75<br>legitimate       |

Are the three flagged transactions equally suspicious?

# Machine Learning Gives Us Predictions

Three are flagged as fraud  
and the three labels are identical

|                                     |                                     |                                          |                                        |
|-------------------------------------|-------------------------------------|------------------------------------------|----------------------------------------|
| Café Central<br>€4.20<br>legitimate | Supermarket<br>€63.10<br>legitimate | Electronics Online<br>€1,299.00<br>fraud | Petrol Station<br>€78.40<br>legitimate |
| Gift Cards ×12<br>€600.00<br>fraud  | Bookshop<br>€24.90<br>legitimate    | Hotel Booking<br>€410.00<br>fraud        | Pharmacy<br>€12.75<br>legitimate       |

Are the three flagged transactions equally suspicious?

Would you block all three cards?

# Machine Learning Gives Us Predictions

Three are flagged as fraud  
and the three labels are identical

|                                     |                                     |                                          |                                        |
|-------------------------------------|-------------------------------------|------------------------------------------|----------------------------------------|
| Café Central<br>€4.20<br>legitimate | Supermarket<br>€63.10<br>legitimate | Electronics Online<br>€1,299.00<br>fraud | Petrol Station<br>€78.40<br>legitimate |
| Gift Cards ×12<br>€600.00<br>fraud  | Bookshop<br>€24.90<br>legitimate    | Hotel Booking<br>€410.00<br>fraud        | Pharmacy<br>€12.75<br>legitimate       |

Are the three flagged transactions equally suspicious?

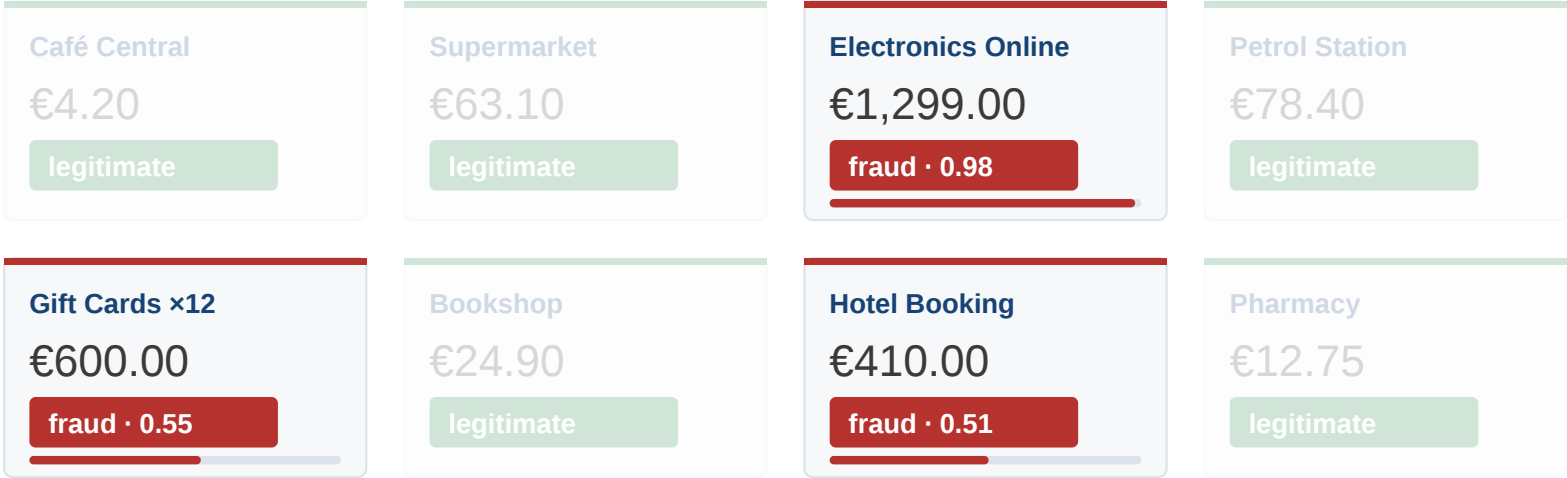
Would you block all three cards?

What would you need to know to treat them differently?

# The Same Batch, With a Number Attached

Now every flagged case carries a number

0.98, 0.55, 0.51 — three very different cases



What can you do now that you could not do a minute ago?

## Speaker notes

Identical picture, identical batch — one click adds the number and the bar, and the three cases that were indistinguishable separate: 0.98 against 0.55 and 0.51. Let students list what they can now do before revealing the cards (P2). Typical answers: rank them, act on the electronics order, send the other two to a person, ask why the model is unsure. Every answer is an action a bare label could not support — the number is not a nicer prediction, it is a **different kind of information**.



# What a Number Lets Us Do

| The number lets us      | What that means for this batch                                                                                    |
|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| Compare and rank        | separate the €1,299 order from the two borderline cases — and compare two models that would both just say "fraud" |
| Weigh consequences      | set the threshold from the cost of a missed fraud against the cost of an annoyed customer                         |
| Defer deliberately      | send 0.55 and 0.51 to a human ask whether more data would move them                                               |
| Combine and communicate | merge evidence from several signals report <i>how sure</i> we are, not only what we think                         |

Speaker notes

The four cards generalise the room's answers; the reveal is the module's central message and should be spoken, not read. Both halves matter: representing uncertainty is the easy part, doing something principled with it is the course.



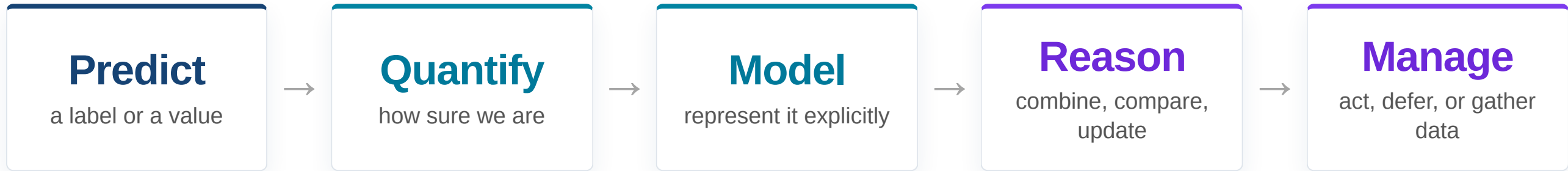
# What a Number Lets Us Do

| The number lets us      | What that means for this batch                                                                                    |
|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| Compare and rank        | separate the €1,299 order from the two borderline cases — and compare two models that would both just say "fraud" |
| Weigh consequences      | set the threshold from the cost of a missed fraud against the cost of an annoyed customer                         |
| Defer deliberately      | send 0.55 and 0.51 to a human ask whether more data would move them                                               |
| Combine and communicate | merge evidence from several signals report <i>how sure</i> we are, not only what we think                         |

**We need to model uncertainty — and we need to be able to reason with it.**



# From Prediction to Managed Uncertainty



And once we want to manage it — **where does the uncertainty actually come from?**

Probabilistic machine learning is not a different way of producing predictions. It is a **language for reasoning about predictions that are not certain.**

Speaker notes

Bridge slide. Read the six cards as one sentence, left to right. The last card is the transition: before we can manage uncertainty we need to know what causes it — and the answer turns out to matter for what we can do about it.



# Mini-Exercise

A loan model gives three applicants a **default probability** of **0.05**, **0.35** and **0.45**.

COST OF ACCEPTING A DEFAULTER

**€8,000**

the loan is not repaid

COST OF REJECTING A GOOD APPLICANT

**€500**

the margin we would have earned

1. Which applicants does the rule "reject if  $p > 0.5$ " reject?
2. **Weigh each option by how likely its bad outcome is.** What is the expected cost of *accepting* each applicant, and of *rejecting* each?
3. At which probability do the two expected costs balance?

# Mini-Exercise — Solution

Fragment the solution. Work the first row aloud: 5% of €8,000 is €400, and 95% of €500 is €475 — so accepting is cheaper, but barely. Then let the break-even line land: it is the whole answer to "how do I weight?".

# Mini-Exercise — Solution

Weigh each outcome by its probability:  $\mathbb{E}[\text{accept}] = p \cdot 8,000$  against  $\mathbb{E}[\text{reject}] = (1 - p) \cdot 500$ .

|   | $p$  | Accept | Reject | Cheaper by | Decision                  |
|---|------|--------|--------|------------|---------------------------|
| A | 0.05 | €400   | €475   | 1.2×       | accept — <i>only just</i> |
| B | 0.35 | €2,800 | €325   | 8.6×       | reject, clearly           |
| C | 0.45 | €3,600 | €275   | 13×        | reject, overwhelmingly    |

**Break-even:**  $p \cdot 8,000 = (1 - p) \cdot 500 \Rightarrow p^* = \frac{500}{8,500} \approx 0.059$



# What the Threshold Tells Us

## THE INTUITIVE RULE

**reject if  $p > 0.5$**

rejects **nobody** here — every applicant is below it, including one whose default risk is nine times the bank's break-even

## THE RULE THE COSTS IMPLY

**reject if  $p > 0.059$**

because a default costs **16×** what a lost customer does

The threshold comes from the **costs**, not from the probability. 0.5 is only the right cut-off when both mistakes are equally expensive — which they almost never are.

- **B and C are not close calls:** rejecting is 8.6× and 13× cheaper.
- **A is the only borderline case** — €400 against €475, and  $p = 0.05$  sits just under  $p^* = 0.059$ . This is where a second signal is worth buying.

## Speaker notes

This slide answers the question students actually have — "where does the threshold come from?" — and corrects the intuition that 0.5 is neutral. Ask what happens to  $p^*$  if the bank's margin doubles ( $p^*$  rises to 0.111, so it accepts more risk), and what happens if the loan is ten times larger ( $p^*$  falls to 0.006). The model output never changed only the costs did.

The borderline case is A, not the 0.35 one: an absolute gap of a few hundred euros means nothing without the scale — what matters is the ratio.



# Takeaway

A prediction without a measure of its uncertainty cannot be compared, weighed against consequences, or handed to a human at the right moment. **Modelling uncertainty and reasoning with it** is what turns a prediction into a decision.

# Where Uncertainty Comes From

---

**Guiding Question:** The model says 0.55. Is that because the world is noisy, or because the model has not learned enough — and would more data help?

# Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

Discussion stepper. Hold the two possibilities apart on the board: "the world is noisy" versus "we have not learned enough". Students usually reach the reducible/irreducible distinction themselves once the third question lands.

# Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

**Is the transaction itself ambiguous** — would a perfect model, with all the data in the world, still be unsure?

# Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

**Is the transaction itself ambiguous** — would a perfect model, with all the data in the world, still be unsure?

**Or has the model simply not seen enough cases like this one?**

# Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

**Is the transaction itself ambiguous** — would a perfect model, with all the data in the world, still be unsure?

**Or has the model simply not seen enough cases like this one?**

**Would collecting more data help** — and could *any* amount of data remove the doubt entirely?

# Two Examples

## EXAMPLE A · AN AMBIGUOUS OBSERVATION

Two patients with **identical** records — same age, same symptoms, same test results. One recovers, one does not.

✗ Would more patients like these help?  
Never.

## EXAMPLE B · AN UNFAMILIAR CASE

A house-price model trained on flats of 40–120 m<sup>2</sup> is asked about a **600 m<sup>2</sup> villa**. It has no examples anywhere near it.

✓ Would ten villas in the training data help?  
Completely.

Which uncertainty can be reduced, and which is part of the problem itself?

Let students classify before the next slide. Push on Example A: "would a better model help?" — no, unless it sees *more attributes* the noise is relative to the information available. That nuance is the honest version of "inherent".

# The Distinction — Then the Names

|                              | Where it comes from                                | More data? | Our example                            |
|------------------------------|----------------------------------------------------|------------|----------------------------------------|
| Uncertainty in the problem   | the outcome genuinely varies given what we observe | ✗ never    | identical patients, different outcomes |
| Uncertainty in our knowledge | we have not seen enough to pin the model down      | ✓ yes      | the 600 m <sup>2</sup> villa           |

Speaker notes

The names arrive last, on a click, after the distinction has been made in plain words (intuition before terminology). Students who can only recite the Greek have learned nothing; students who can sort new cases into the two columns have learned the concept.



# The Distinction — Then the Names

|                              | Where it comes from                                | More data? | Our example                            |
|------------------------------|----------------------------------------------------|------------|----------------------------------------|
| Uncertainty in the problem   | the outcome genuinely varies given what we observe | ✗ never    | identical patients, different outcomes |
| Uncertainty in our knowledge | we have not seen enough to pin the model down      | ✓ yes      | the 600 m <sup>2</sup> villa           |

## Aleatoric

*from alea, dice* — noise in the world

## Epistemic

*from episteme, knowledge* — ignorance in the model

# Why the Distinction Matters

| If the uncertainty is...               | The right response                                                   | The expensive mistake                        |
|----------------------------------------|----------------------------------------------------------------------|----------------------------------------------|
| <b>in our knowledge</b><br>(epistemic) | collect data where the model is thin defer to a human who knows more | accepting it as inherent and never improving |
| <b>in the problem</b><br>(aleatoric)   | model the noise set a policy for the ambiguous cases                 | buying more data that cannot help            |

A model that can tell the two apart can report that it is **out of its depth**, not merely unsure.

A probabilistic model can represent **both** kinds of uncertainty — and can often tell you which one it is facing. That is why the distinction is worth a name.

# Takeaway

Uncertainty has two sources: **noise in the world** and **gaps in our knowledge**. Only the second shrinks with data, and a decision — collect more, defer, act — depends on knowing which one you face.

## Speaker notes

This is the consequence slide the terminology was for. It is also the bridge out: we now know uncertainty matters and comes in kinds — but we still have no systematic way to write it down, combine it or update it. That is the next module.



# Beyond True or False

---

**Guiding Question:** A proof either holds or it does not. A learned prediction is never a proof — so what kind of conclusion is it, and how strong is it?

# Three Modes of Reasoning

|                                       | From              | To                                    | What it guarantees                       |
|---------------------------------------|-------------------|---------------------------------------|------------------------------------------|
| <b>Deduction</b> <i>(logic)</i>       | premises          | what <b>must</b> follow               | certain — but adds no new content        |
| <b>Induction</b> <i>(statistics)</i>  | observed patterns | a <b>regularity</b> , a hypothesis    | nothing — content-adding, not guaranteed |
| <b>Abduction</b> <i>(probability)</i> | observations      | the <b>best available explanation</b> | nothing — only comparable against rivals |

Machine learning lives in the bottom two rows.



Three modes, one criterion to compare them: what does each guarantee, and what does it buy? Deduction guarantees everything and discovers nothing induction and abduction discover and guarantee nothing. Data science lives in the latter two, which is why validation is a permanent obligation.

# The Scientific Reasoning Cycle

The three modes are not alternatives — they take turns.

**Example: gravity.**

## Deduction

From the theory: all objects of equal mass fall at the same rate everywhere on earth.

## Induction

Design an experiment, measure fall times at several locations, compute means and standard deviations.

## Abduction

Times agree at one location but vary between locations.  
**Best explanation:** distance from earth's mass centre.

The cycle turns: the abducted explanation becomes the next theory to deduce from. A data-science project runs the same loop, usually several times.

Speaker notes

Have students place a machine-learning project in the cycle: choosing a model family is deduction from assumptions, fitting is induction, and diagnosing why it fails on a subgroup is abduction. The loop is the honest picture of the workflow, unlike the tidy pipeline diagrams.



# Mini-Exercise

A dashboard shows that customers who use the mobile app churn **less** than those who do not.

- 1 What would a purely **inductive** reading conclude?
- 2 Give two different **abductive** explanations.
- 3 What **deduction** would let you distinguish them?

## Speaker notes

Question and solution share a slide: the three prompts stay visible while the answers appear, so students can check each answer against the question that produced it. One click reveals all three.

The exercise rehearses the whole cycle in a business setting and previews the correlation/causation problem. The key move is 3: a hypothesis earns its keep by predicting something checkable that its rival does not.



# Mini-Exercise

A dashboard shows that customers who use the mobile app churn **less** than those who do not.

- 1 What would a purely **inductive** reading conclude?
- 2 Give two different **abductive** explanations.
- 3 What **deduction** would let you distinguish them?

## INDUCTION

"App use reduces churn" — a regularity read straight off the data.

## ABDUCTION

The app causes loyalty — *or* loyal customers install the app. Same correlation, opposite causal direction.

## DEDUCTION

If the app causes loyalty, randomly prompting a subgroup to install it should lower their churn — an experiment deduced from the hypothesis.

# Beyond True or False

|                        | Classical reasoning       | Probabilistic reasoning           |
|------------------------|---------------------------|-----------------------------------|
| <b>Conclusions</b>     | follow, or do not         | are supported to <i>degrees</i>   |
| <b>Evidence</b>        | premises, taken as given  | may be partial or noisy           |
| <b>Certainty</b>       | the conclusion is certain | the conclusion may stay uncertain |
| <b>New information</b> | can only add premises     | can <i>change</i> what we believe |

Classical logic asks *whether* a conclusion follows. Probabilistic reasoning also asks **how strongly the evidence supports each possible conclusion**.

Connect to the three modes of reasoning just seen: deduction is the left card; induction and abduction live on the right, and they are the modes machine learning actually performs. A learned prediction is never a theorem — it is an inference from evidence, and its strength is the thing we want to write down.

# What the Language Has to Do

We now know that uncertainty matters and that it has different sources. What we still lack is a **systematic way** to:

- 1 **Represent** — say how much we believe each possibility
- 2 **Combine** — put several uncertain pieces of information together
- 3 **Update** — change our beliefs when new evidence arrives
- 4 **Decide** — act on beliefs that are not certainties

# What the Language Has to Do

We now know that uncertainty matters and that it has different sources. What we still lack is a **systematic way** to:

- 1 **Represent** — say how much we believe each possibility
- 2 **Combine** — put several uncertain pieces of information together
- 3 **Update** — change our beliefs when new evidence arrives
- 4 **Decide** — act on beliefs that are not certainties

**Probability provides that language.** The formal material that follows exists because each of these four operations turns out to have exactly one consistent set of rules.

# Takeaway

Machine-learning predictions are inferences from evidence, not derivations from premises.  
**Probability is the calculus of such inferences** — which is why the next lecture is about it, and why it will feel like an answer rather than a prerequisite.

The reveal is the pivot of the whole introduction: the mathematics is now the solution to a problem the room has already felt. Say explicitly that "exactly one consistent set of rules" is a real theorem (Cox), not a slogan — but leave it there.

# What Probabilistic ML Enables

---

**Guiding Question:** Which questions about a machine-learning system will you be able to answer after this course that you cannot answer today?

# After This Course, You Will Be Able to Ask — and Answer

## ABOUT A PREDICTION

- ? How certain is it, and should I trust it?
- ? When should the model refuse to decide?
- ? What alternatives are plausible?

## ABOUT THE DATA

- ? When would more data help?
- ? When is uncertainty unavoidable?
- ? What does this data actually justify?

## ABOUT LEARNING

- ? How should belief change when evidence arrives?
- ? How do I combine prior knowledge with observations?
- ? How do I compare competing explanations?

## ABOUT THE METHODS

- ? Why do the common methods work, and under which assumptions?
- ? When should I expect them to fail?
- ? What is my model assuming without telling me?

Utility before curriculum structure. These are not learning objectives to be graded they are the questions the students should recognise as *theirs* by the end. Ask which of them they can answer today — usually none with confidence.

# What Changes Compared to Traditional Machine Learning

## TRADITIONAL ML ASKS

What is the prediction?

## PROBABILISTIC ML ALSO ASKS

- ? **How certain** is it?
- ? **What alternatives** are plausible?
- ? **Why** are we uncertain?
- ? **How does new evidence** change what we believe?
- ? **What decision** follows from this?
- ? **What can and cannot** be inferred from this information?

# Takeaway

Probabilistic machine learning does not replace prediction. It **surrounds every prediction with the questions a decision needs answered** — and gives one language in which all of them can be asked.

Closing frame for the motivation half of the lecture. The right-hand card is the course in six questions; each later lecture answers one or two of them, which the roadmap at the end will make explicit.

# How Machines Learn

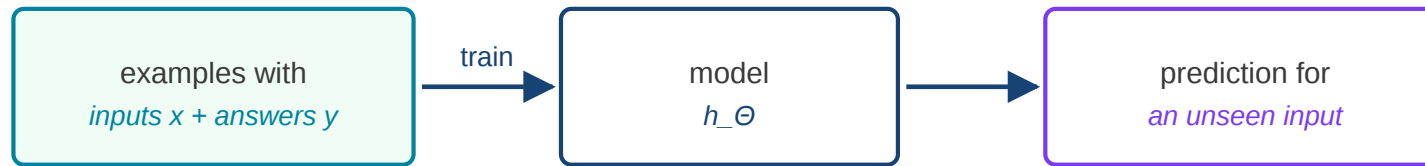
---

**Guiding Question:** Nobody writes the rule. So what exactly is a machine given, what does it build, and where does uncertainty get in?

# Learning From Answers

A radiologist has labelled ten thousand scans. A bank has ten years of loans and knows which ones defaulted. In both cases **every example carries its answer**.

Supervised: every training example carries the answer



The learner is told the correct output for each case it sees.

What information does the learner receive during training?

## Speaker notes

Ask before the click: what exactly does the learner see? The answer — an input *and* the output we wanted — is what makes this the easiest setting, and the most expensive one, because somebody had to produce every label.

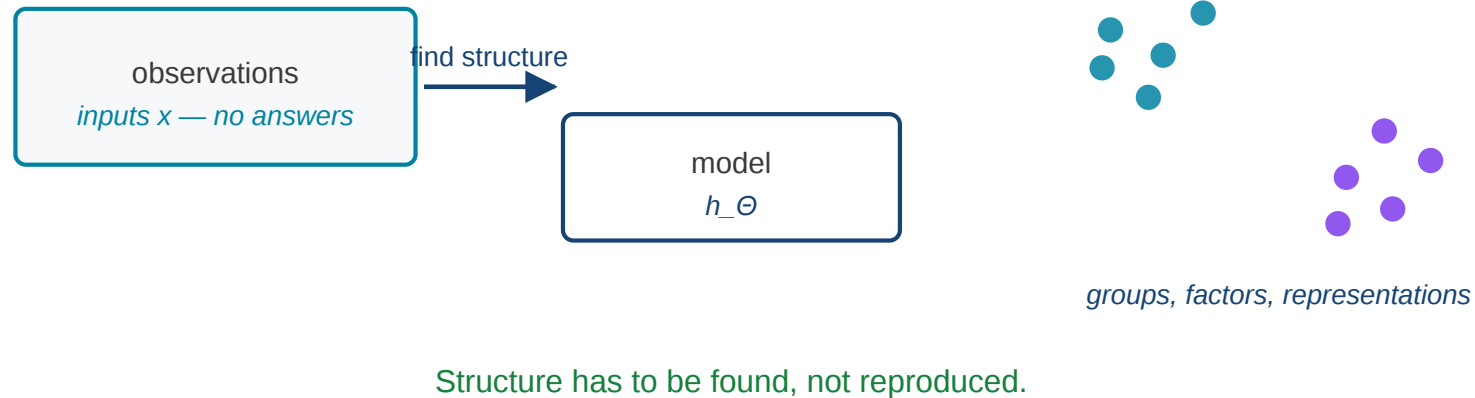
Good examples to name: image classification, medical diagnosis, house prices. The last one previews regression: the "answer" need not be a class.



# Learning Without Answers

Now the same bank wants to know **what kinds of customer it has**. Nobody has ever written down the list — the categories are what we are looking for.

Unsupervised: no answers — only the observations themselves



What can we learn when nobody tells us the correct answer?

## Speaker notes

Let the room answer first: groups of similar customers, the few dimensions that carry most of the variation, what is unusual. Name clustering, dimensionality reduction, latent structure — but only after they have said it.

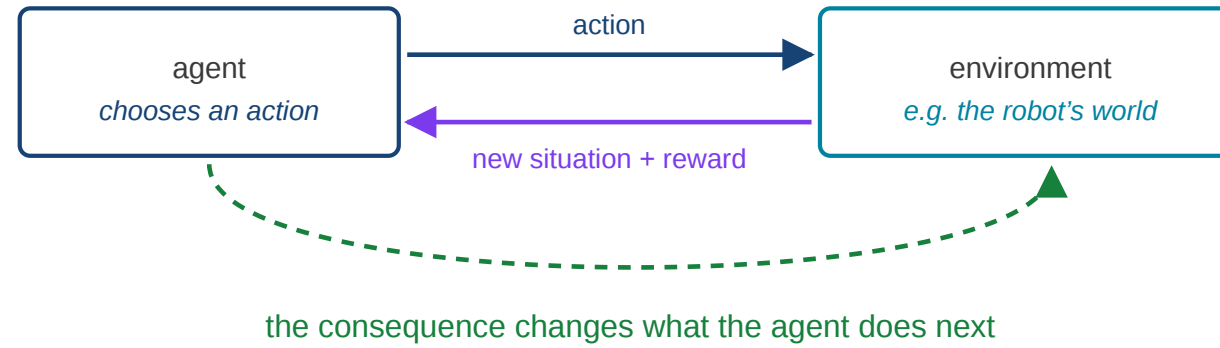
The honest difficulty to flag: without an answer key there is no unambiguous notion of "correct", so evaluation becomes a genuine problem. That returns in the clustering lecture.



# Learning by Acting

A robot has to cross a room. Nobody can label the correct motor command for every situation it might find itself in — and it only discovers whether a move was good **after making it**.

Reinforcement: no answer is given — only the consequence of acting



The learner must act to find out whether the action was good.

How can a learner improve when it must act in order to find out whether the action was any good?

Step 1 draws action and consequence; step 2 closes the loop back into the agent's behaviour. Only now name the vocabulary: **agent**, **environment**, **action**, **reward**.

The point that distinguishes this from supervised learning: the feedback says how good the outcome was, not what the right action would have been — and it may arrive long after the action that caused it.

# Same Goal, Different Information

Same goal, different information

|               | WHAT IT IS GIVEN         | WHAT FEEDBACK IT GETS  |
|---------------|--------------------------|------------------------|
| Supervised    | inputs + answers         | the answer, every time |
| Unsupervised  | inputs only              | nothing                |
| Reinforcement | the situation it acts in | a reward, after acting |

Reveal one row per click, so the comparison is assembled from the three examples the students have just seen rather than presented as a taxonomy.

Ask which paradigm a spam filter uses (supervised), a recommender trained on clicks (arguably all three), a game engine (reinforcement). The boundaries are softer than the table suggests, and saying so is more honest than a clean three-way split.

# Same Goal, Different Information

Same goal, different information

|               | WHAT IT IS GIVEN         | WHAT FEEDBACK IT GETS  |
|---------------|--------------------------|------------------------|
| Supervised    | inputs + answers         | the answer, every time |
| Unsupervised  | inputs only              | nothing                |
| Reinforcement | the situation it acts in | a reward, after acting |

The paradigms differ in **what the learner is given** and **what feedback it gets** — not in what we ultimately want from it.

# What Is Common to All Three?

## Nobody wrote the rule

The mapping from situation to answer is **too complex or too implicit** to specify by hand.

## Examples exist

There is data — or experience — in which the desired behaviour is demonstrated, even if it was never described.

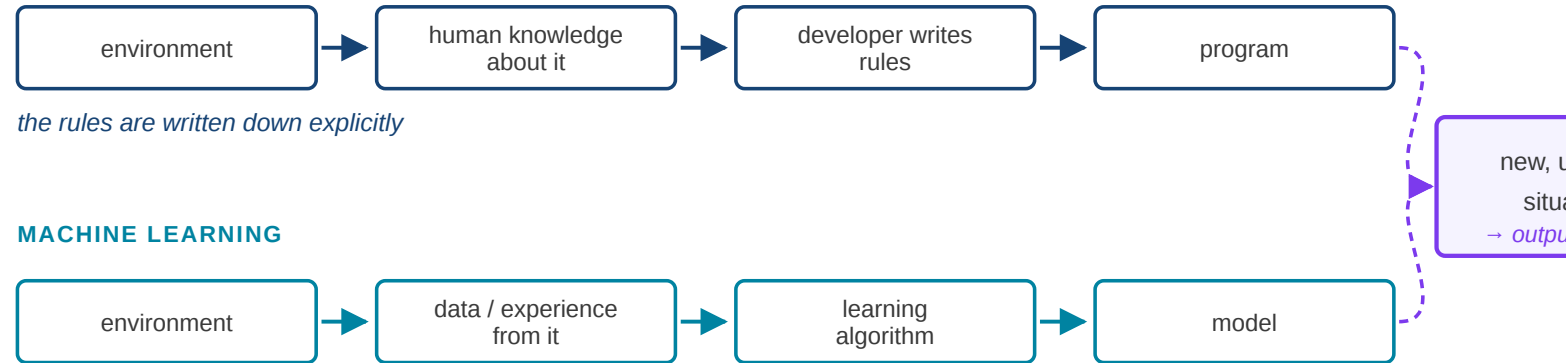
So what replaces the rule that nobody wrote?

This is the bridge to the next slide. Hold the question: in all three paradigms, something is derived from data that a developer would otherwise have had to write down. That "something" is the model — and comparing the two ways of getting it is the next slide.

# Two Ways to Get Behaviour

Both are judged on situations neither of them has seen

## TRADITIONAL PROGRAMMING



*the rules are written down explicitly*

## MACHINE LEARNING



*the rules are inferred from what the environment produced*

## Speaker notes

Three clicks. First the programming pipeline: the developer holds knowledge about the environment and writes it down as rules. Second the learning pipeline: the environment produces data, and an algorithm derives the model. Third — the point of the slide — **both arrive at the same destination**: a situation neither the developer nor the learner has seen.

Ask between clicks two and three: what is the same in both rows? The environment on the left, and the demand on the right.



# What Actually Changed

## TRADITIONAL PROGRAMMING

The developer's **knowledge becomes rules**. Deduction: rules plus input give the output.

*The assumptions are visible — they are in the code.*

## MACHINE LEARNING

The environment's **data becomes a model**. Induction: examples plus a learning procedure give a rule.

*The assumptions are still there — but now they are in the data, the model family and the objective.*

The rules do not disappear when nobody writes them. They become **harder to inspect** — which is why the rest of this lecture is about making every part of that pipeline explicit.

Speaker notes

Deliberate callback to the three modes of reasoning: programming is deductive, learning is inductive. Students who saw that module will recognise the shape, and the guarantee that came with each.

The warning is the honest framing of the trade and sets up the design questions at the end of the module.



# What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

Questions first, names second (P4). Let the room produce the three questions in their own words — they always do — and only then attach Mitchell's (1997) vocabulary. A definition that arrives as a label for something already understood costs nothing to remember.

# What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

## **Task $T$**

what the system should learn  
to do

## **Experience $E$**

what it learns from

## **Performance $P$**

how we decide it improved

# Chess



A program that plays, loses, and plays again — millions of times against itself.

- **Task** — play chess well
- **Experience** — games played against itself, and positions from expert play
- **Performance** — fraction of games won; rating against other engines

Speaker notes

The classic example, and worth one observation: the experience here is *generated by the system itself*. No one labelled the good moves — the outcome of the game did.



# A Large Language Model



A model shown enormous quantities of text, asked only to continue it.

- **Task** — predict the next word
- **Experience** — a very large corpus of text, then human feedback
- **Performance** — likelihood of held-out text; later, human judgement

Two systems with nothing in common — a board game and a text corpus — fit the **same three-part description**. That is what makes it a definition rather than a description of one program.

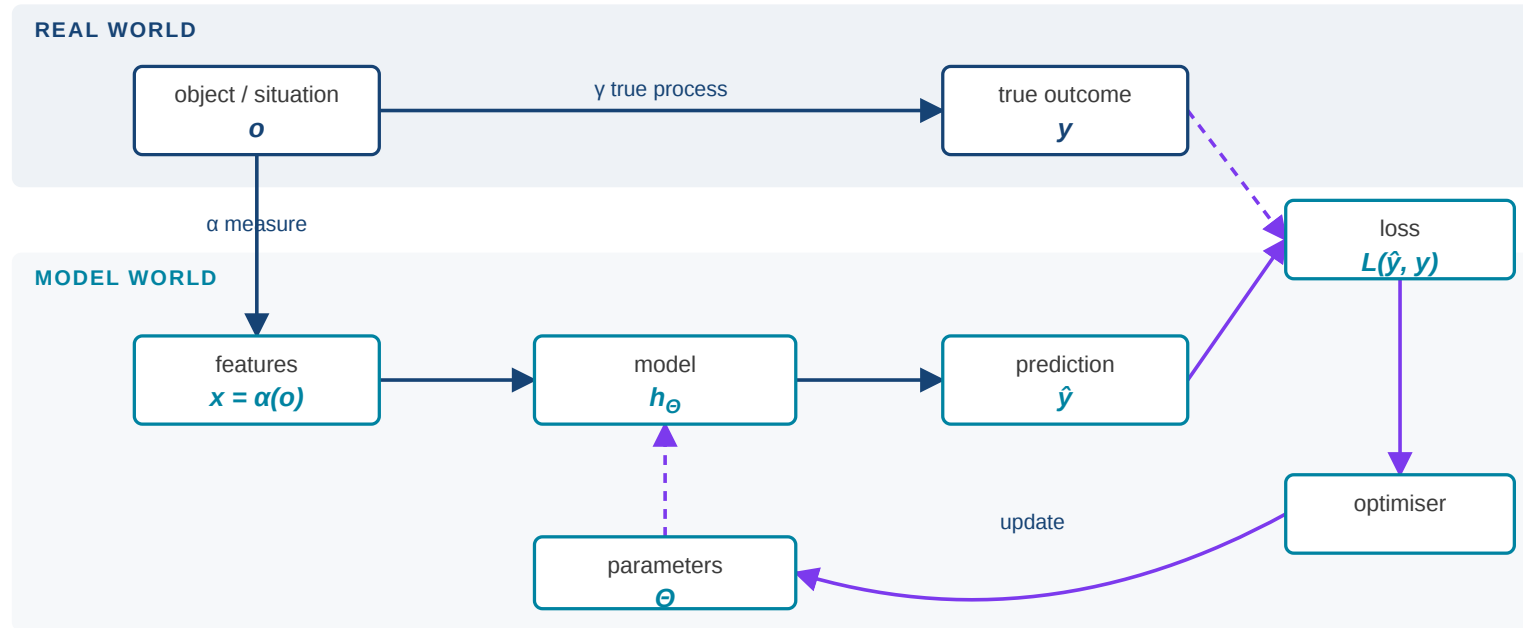
## Speaker notes

Pause on the task: "predict the next word" looks far too weak to produce what these systems do. That is exactly why the choice of a *proxy* task is a real design decision — it must force the model to represent what actually matters.



# What Would We Ideally Like to Know?

The optimiser changes the parameters — and the training loop closes



There is some real process that turns a situation into an outcome. Why can we not simply write it down?

## Speaker notes

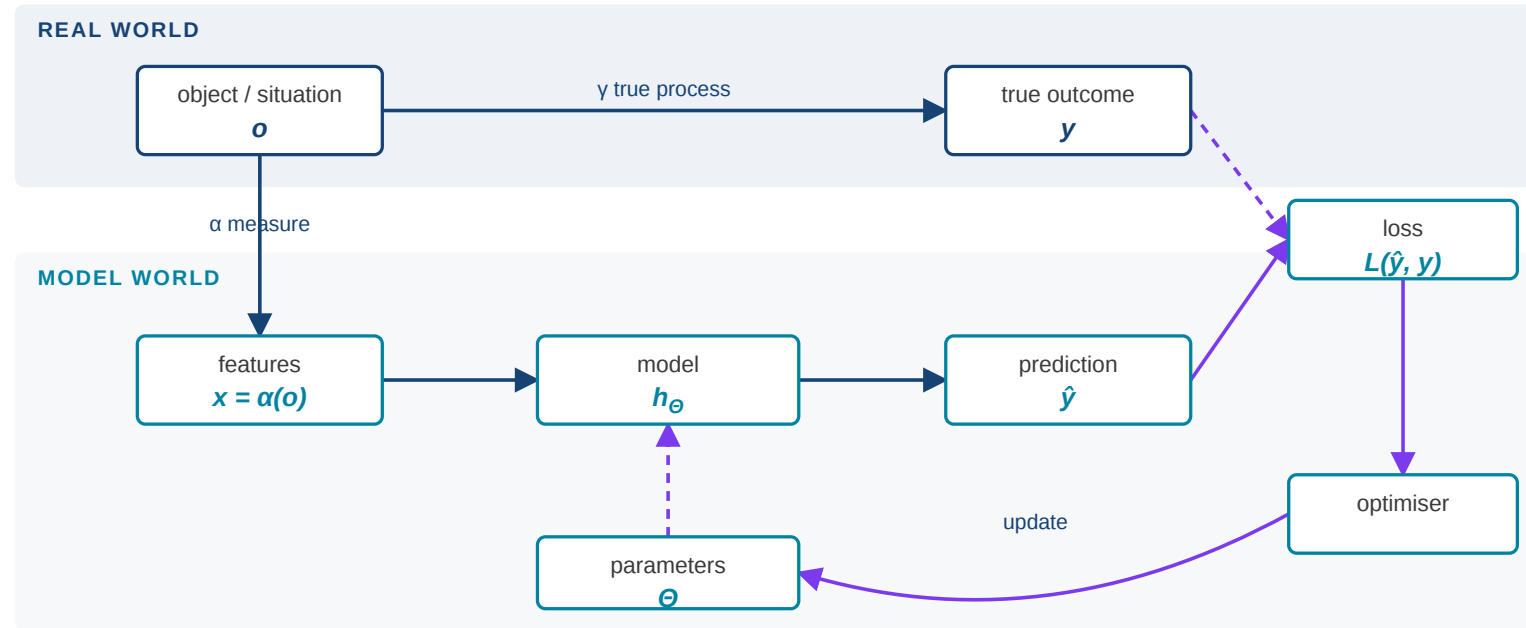
Only the real-world band is drawn: an object or situation  $o$ , the true process  $\gamma$ , the outcome  $y$ . Let the room answer the question — the process is unknown, or too complex, or not accessible to us at all. That inaccessibility is the reason everything that follows exists.

Notation, kept from the source:  $\gamma$  **is the ideal target function** on real-world objects.



# From the Real World to a Model of It

The optimiser changes the parameters — and the training loop closes



# Two Worlds, One Correspondence

$$\gamma(o); \rightsquigarrow; h_{\Theta}(\alpha(o)); \equiv; h_{\Theta}(x)$$

$\gamma$

the **real process** — unknown,  
and not ours to write down

$\alpha$

the **representation** — what  
we chose to measure

$h_{\Theta}$

the **model** — our  
approximation, with  
parameters  $\Theta$

We never learn the world. We build a **model world** and approximate the process inside it.

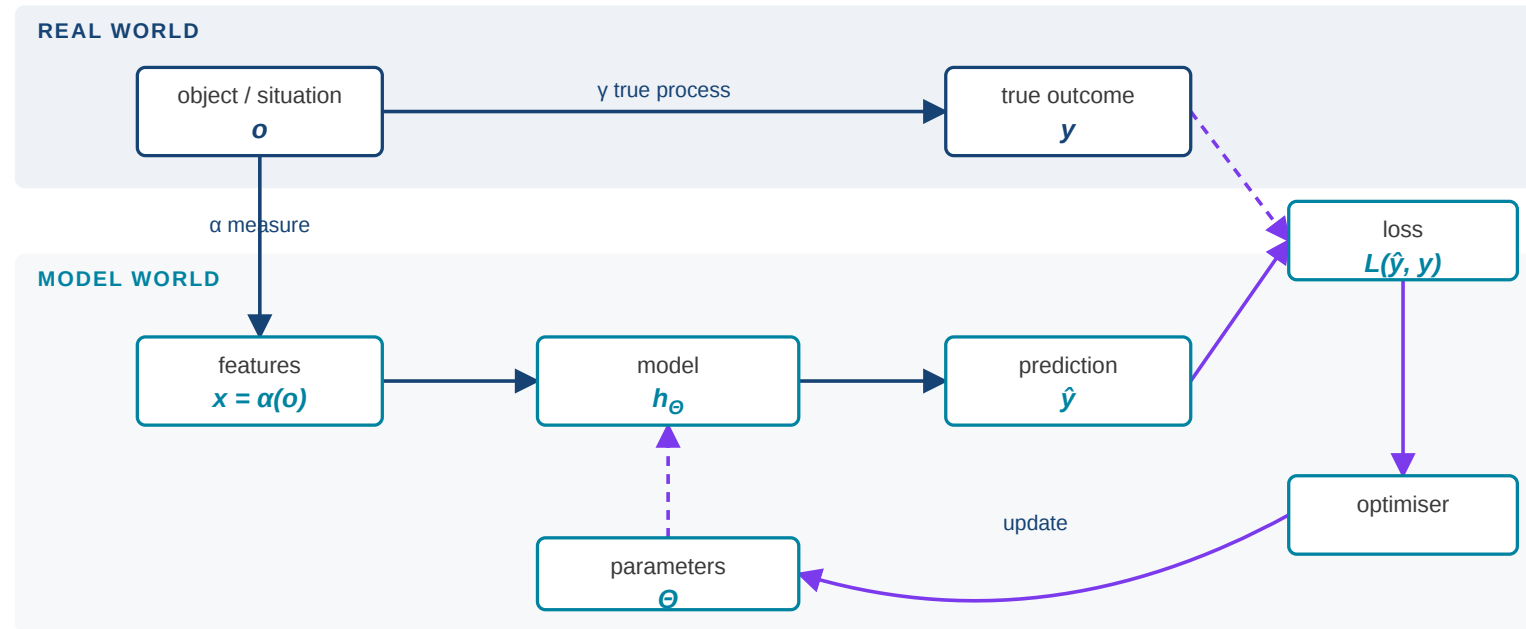
Two clicks: first the measurement — the **model-formation function**  $\alpha$  turns an object into features  $x = \alpha(o)$  — then the model  $h_{\Theta}$  and its prediction.

Be precise about the notation, because it is easy to garble:  $\gamma$  is the real process,  $\alpha$  **is the representation**,  $h_{\Theta}$  is the model and  $\Theta$  are its parameters.  $\alpha$  is *not* the parameters.

The relation on the fragment is the heart of the module: two worlds, and a correspondence between them that is only ever approximate. Most practical failures are failures of  $\alpha$  — the features did not carry what  $\gamma$  depends on.

# Closing the Loop

The optimiser changes the parameters — and the training loop closes



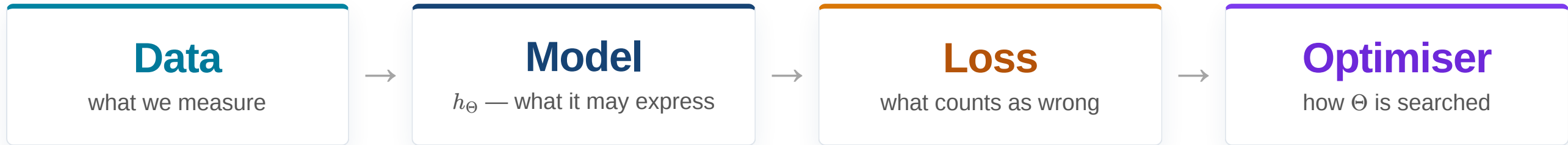
## Speaker notes

Two more clicks complete the figure. Click 3 adds the **loss**, which is the first thing on the slide that compares the model world with the real one: it needs  $\hat{y}$  *and* the outcome  $y$  we actually wanted. Click 4 adds the **optimiser**, which changes the **parameters**  $\Theta$  — and the loop closes.

Read it aloud once as a sentence: measure the situation, predict, compare with what happened, adjust, repeat. Every method in this course fills these boxes differently.



# The Four Choices

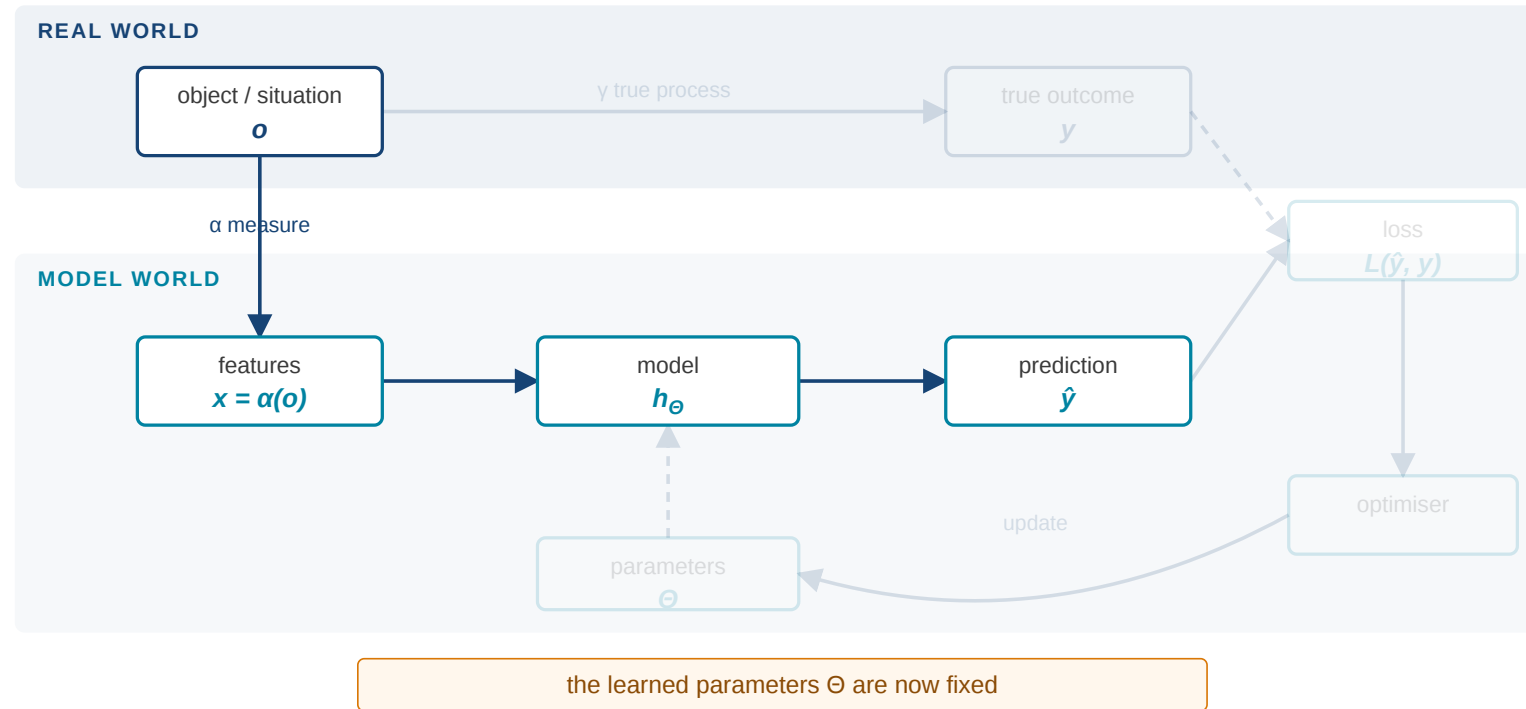


The optimiser has settings of its own that are not learned from the data — **hyperparameters**. Choosing them is part of the outer process, not of the loop.

The compact restatement of the figure, and the map for the whole course: every algorithm later is a different set of these four. Have students place linear regression into the four boxes before moving on.

# Training Is Not Using

Inference: a new, unseen situation follows one path — the training machinery is gone



Which of these boxes still exist when the system is actually deployed?

## Speaker notes

One click dims everything the training loop needed. What remains is a single path: a new situation, measured, through the model, to a prediction — with  $\Theta$  now fixed.

Ask the question before clicking. Students routinely assume the loss is somehow still "checking" the model in production; it is not, because in production nobody tells us  $y$ . That gap — no answers after deployment — is what monitoring and the evaluation methodology have to work around.



# The Loop Around the Loop

Data, model, loss and optimiser describe **what runs**. They say nothing about how anybody decided on them.

## INNER LOOP · THE LEARNING SYSTEM

data → model → prediction → loss →  
optimiser → updated parameters

*runs thousands of times per minute,  
automatically*

## OUTER LOOP · THE DEVELOPMENT PROCESS

collect → represent → split → train →  
validate → diagnose → revise → evaluate  
→ deploy → monitor

*runs a handful of times per project, and  
every step is a human decision*

## Speaker notes

The distinction is the point of the slide. The inner loop is an optimisation; the outer loop is engineering and scientific method. A student who only ever sees the inner loop concludes that machine learning is "fitting", which is the part a library already does for them.



# What the Outer Loop Decides

## BEFORE TRAINING

- what to collect, and from where
- how to clean and represent it
- how to split train / validation / test

## WHILE DEVELOPING

- which model family, which hyperparameters
- which errors to look at, and why they happen
- what to change next

## AFTER TRAINING

- is it good enough, and by whose measure
- does it hold up on data it has never seen
- is it still holding up next month

Machine learning is **experimental**: you make choices, then test whether they worked. Measured performance is *evidence* about future performance — never a guarantee of it.

## Speaker notes

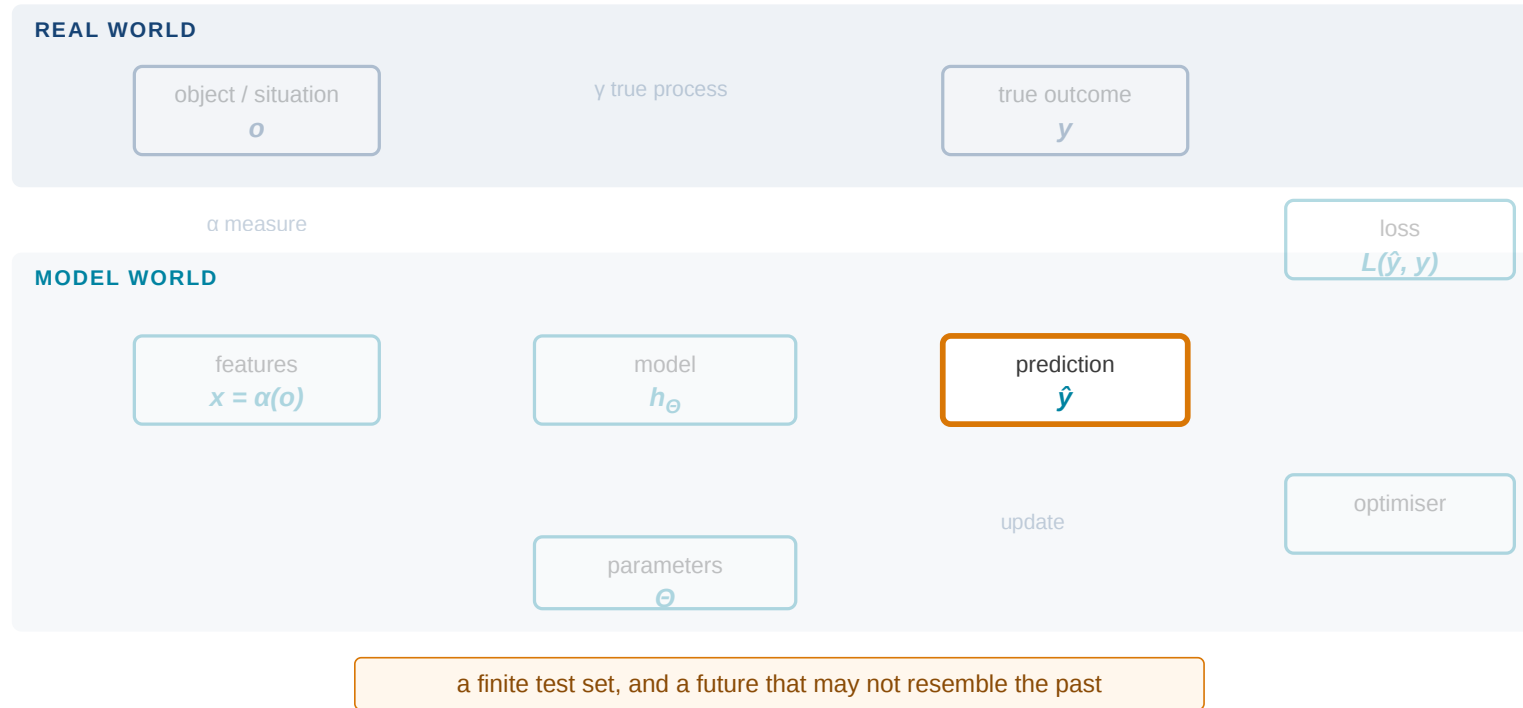
The insight is the bridge to probability. Every number produced by this outer loop is an estimate from a finite sample: a validation score is a measurement with error bars, not a property of the model.

Ask what happens when the test set has been consulted fifty times during development — it stops being held out, and the estimate stops being honest.



# The Same System, Read for Uncertainty

In the evaluation



The same figure the module has been building, now read a second way. One click per source, each highlighting the box it lives in:

1. **the world** — identical situations, different outcomes (aleatoric)
2. **the data** — noisy measurement, missing values, an unrepresentative sample
3. **the model** — the family may not contain the truth
4. **the parameters** — too few examples to pin them down (epistemic; this is the one more data reduces)
5. **the objective** — the loss is a *design choice*; it may not capture the task
6. **the optimisation** — stochastic steps, initialisation, approximate solutions
7. **the evaluation** — a finite test set, and a future that may differ

Ask before each click: where would *you* look next?

# Two Distinctions Worth Keeping Straight

## UNCERTAINTY *IN* THE LOSS

The quantities entering the loss are uncertain:  $y$  is noisy, the sample is finite. The loss itself is an ordinary function.

## UNCERTAINTY *ABOUT* THE LOSS

Whether this objective represents the task at all. That is a modelling judgement, not a random variable.

Do not say "the loss is random". Say what is uncertain: the data it is computed from, or our confidence that it is the right objective.

The first pair matters because students conflate them and then write things that are false. The closing statement is the return to the course's opening claim, now with a system diagram behind it rather than a fraud example.

# Two Distinctions Worth Keeping Straight

## UNCERTAINTY *IN* THE LOSS

The quantities entering the loss are uncertain:  $y$  is noisy, the sample is finite. The loss itself is an ordinary function.

## UNCERTAINTY *ABOUT* THE LOSS

Whether this objective represents the task at all. That is a modelling judgement, not a random variable.

Do not say "the loss is random". Say what is uncertain: the data it is computed from, or our confidence that it is the right objective.

Uncertainty is not a property of the final prediction. It **enters throughout the system** — and probabilistic machine learning is the language for representing and reasoning about it at every one of these places.

# Designing the Learning Problem

Before any model is chosen, four things have to be decided — and each of them is a claim about the world.

## THE TASK

- ? What should the system predict or decide?
- ? What is one unit of prediction?
- ? What counts as success?

## THE DATA

- ? Which observations represent the problem?
- ? How were the examples sampled?
- ? Will future cases look like these?

## THE REPRESENTATION A

## THE TARGET



? What are the inputs, concretely?

? Which features are available — and which are merely convenient?

? What is lost in the measurement?

? What is  $\gamma$ , in words a domain expert would accept?

? Is the label we have the thing we care about, or a proxy for it?

Every box is a modelling decision that the mathematics will later take for granted. The proxy question in the last box is the one with the most consequences in practice: "did the patient have a disease" versus "was the patient diagnosed", "is this content good" versus "was it clicked".

# Designing the System

## MODEL AND OBJECTIVE

- ? What does the model family assume, and how flexible is it?
- ? Does minimising this loss correspond to what we actually want?

## LEARNING

- ? Which procedure searches the parameters?
- ? How sensitive is the result to initialisation and randomness?

# Designing the System — Evaluation

## EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

## UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

Close the module by pointing at the questions rather than at topics — the roadmap on the next slide then reads as answers to them. The last box is deliberately the one that returns to Lecture 01's opening: a prediction exists to support a decision.



# Designing the System — Evaluation

## EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

## UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

These are the questions the rest of the course answers. Each lecture takes one or two of them and makes them precise.

# The Road Ahead

---

**Guiding Question:** Now that the motivation is in place — which lecture answers which of your questions?

# The Road Ahead

Each lecture answers one of the questions you have just met.

**02**

**Representing uncertainty** *How do we describe uncertainty mathematically?*

Probability, distributions, and how to summarise data.

**03**

**Learning from evidence** *Why do the classical methods work — and what do they assume?*

Linear and logistic regression, re-read as probabilistic models.

**04**

**Combining information** *How do uncertain quantities relate — and when can we treat them separately?*

Naive Bayes, independence, and the graphical view.

**05**

**Structured models** *How do we write down a large, structured set of beliefs?*

Bayesian networks, fitting them, and drawing them.

**06**

**Judging models** *What is the quantity we actually want to minimise — and what did we get instead?*

Evaluation and empirical risk minimisation.

**07**

**Deciding and refining** *How do we keep uncertainty all the way to the decision?*

Mixtures, Bayesian regression, and what comes after.

Only now the detailed structure — the students can read each question as one of their own. Point at the arc: represent → learn → combine → structure → judge → decide.



# The Programme, in Three Moves

Every method in this course is the same three moves, in the language you are about to learn:

## Model the joint

Write down what the world would have to be like to produce what we observed — the data *and* the unknowns together.

## Condition on the data

Inference is conditioning: fix what was observed, and let the rules of probability say what follows for the rest.

## Read off answers

Predictions, uncertainties and decisions are all quantities computed from that one conditioned model.

Every one of these words — joint, condition, inference — gets a precise meaning in Lecture 02. They appear here only so that you recognise them when they return.

Speaker notes

This is the former "programme" slide, moved to the end where its vocabulary is motivated rather than a burden. Return to it at the start of every later lecture and mark which move the lecture is making.



